

1. Vorislik (Inheritance)

Ta'rif:

Vorislik — bu mavjud klass asosida yangi klass yaratish imkoniyatini beradigan mexanizm. Bunda yangi klass (subclass, child) ota-klass (superclass, parent) metodlari va atributlarini meros qilib oladi.

Sintaksis:

```
class Parent:
    # ota klass kodi
    pass

class Child(Parent):
    # bola klass kodi
    pass
```



Afzalliklari:

- Kodni qayta ishlatish imkoniyati
- Real hayotdagi ierarxiyalarni model qilish (masalan: `Person` -> `Employee`)
- Metodlarni qayta aniqlash (overriding) imkoniyati
- Polimorfizm va kengaytiriladigan dizayn

Oddiy misol:

```
class Person:
    def __init__(self, name):
        self.name = name

    def display(self):
        print(f"Name: {self.name}")

class Employee(Person):
    def work(self):
        print(f"{self.name} works")

tom = Employee("Tom")
tom.display()    # Name: Tom
tom.work()       # Tom works
```



Amaliy mashqlar (Vorislik)

1. `Animal` nomli oddiy klass yarating. Unda `sound()` metodi bo'lsin va umumiy ovoz chiqaradigan qilib yozing. Keyin `Dog` nomli klass yarating va uni `Animal` klassidan voris

qiling. Dog klassida `sound()` metodini qayta yozing (override) va u "Woof!" deb chiqarsin. Dog klassidan obyekt yaratib, uning metodini chaqiring.

Kod:

```
class Animal:
    def sound(self):
        print("Some generic animal sound")

class Dog(Animal):
    def sound(self):
        print("Woof!")

# Sinov
dog = Dog()
dog.sound()    # Woof!
```

2. Person nomli klass yarating. U odamlarni ifodalashtirsin. Keyin undan Student klassini voris oling. Student klassiga `study()` metodi qo'shing va u "I am studying" xabarini chiqarsin. Student obyektidan foydalanib, metodni chaqirib ko'ring.

Kod:

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def study(self):
        print(f"{self.name} is studying")

# Sinov
student = Student("Ali")
student.study()    # Ali is studying
```

3. Person klassini yarating va u odamning name atributini saqlasin. Keyin Employee nomli klass yarating va uni Person klassidan voris qiling. Employee klassida qo'shimcha ravishda salary atributini qo'shing. Obyekt yaratib, name va salary ni chiqaring.

Kod:

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, salary):
```

```

        super().__init__(name)
        self.salary = salary

# Sinov
employee = Employee("Dilshod", 5000)
print(employee.name)    # Dilshod
print(employee.salary)  # 5000

```

4. Klasslar ierarxiyasini yarating:

- Person klassi name saqlasin.
- Employee klassi Person dan voris olsin va unga salary qo'shilsin.
- Manager klassi esa Employee dan voris olsin va unga department atributi qo'shilsin.

So'ngra Manager obyektini yaratib, name, salary va department ma'lumotlarini chiqaring.

Kod:

```

class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, salary):
        super().__init__(name)
        self.salary = salary

class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department

# Sinov
manager = Manager("Zafar", 8000, "IT")
print(manager.name)    # Zafar
print(manager.salary)  # 8000
print(manager.department) # IT

```

Maxsus metodlar (dunder methods)

Python'dagi **maxsus metodlar** (yoki **dunder methods** — double underscore methods) — bu klasslarga **maxsus xatti-harakatlarni** qo'shish imkonini beradigan metodlardir. Ular har doim ikki pastki chiziq bilan boshlanib tugaydi, masalan:

- `__init__` → konstruktor, obyekt yaratilganda ishga tushadi
- `__str__` → obyektning matn ko'rinishida qaytaradi (`print()` chaqirilganda)
- `__repr__` → obyektning texnik ko'rinishi (debug uchun)
- `__len__` → `len(obj)` chaqirilganda ishlaydi

- `__getitem__` → indeks bo'yicha obyekt elementini olish (`obj[0]`)
- `__setitem__` → indeks bo'yicha qiymatni o'zgartirish (`obj[1] = x`)
- `__delitem__` → elementni o'chirish (`del obj[2]`)
- `__iter__`, `__next__` → obyektни iterator qilish (`for` siklda ishlatish)
- `__contains__` → `in` operatorini ishlatish uchun
- `__eq__`, `__lt__`, `__gt__` → taqqoslash operatorlari (`==`, `<`, `>`)
- `__add__`, `__sub__`, `__mul__`, `__truediv__` → arifmetik operatorlar uchun
- `__call__` → obyektни funktsiya kabi chaqirish imkoniyati
- `__enter__`, `__exit__` → kontekst menejer (`with` operatori)

◆ Vazifasi

Maxsus metodlar yordamida biz oddiy funktsiyalar va operatorlarning klass obyektlariga **tabiiy ishlashini** ta'minlaymiz.

Ya'ni:

- `print(obj)` → avtomatik `__str__()` metodini chaqiradi.
- `len(obj)` → avtomatik `__len__()` metodini chaqiradi.
- `obj1 + obj2` → avtomatik `__add__()` metodini chaqiradi.
- `obj[i]` → avtomatik `__getitem__()` metodini chaqiradi.
- `obj1 == obj2` → avtomatik `__eq__()` metodini chaqiradi.

◆ Asosiy turlari va vazifalari

1. Obyekt yaratish va o'rnatish

- o `__init__(self, ...)` – konstruktor, obyekt yaratilganda ishlaydi.
- o `__del__(self)` – destruktur, obyekt o'chirayotganda ishlaydi.

2. Matnli ko'rinish

- o `__str__(self)` – `print(obj)` chaqirilganda ishlaydi.
- o `__repr__(self)` – obyektни aniqroq matnli ko'rinishda qaytaradi (`debug` uchun).

3. O'lcham va elementlarga murojaat

- o `__len__(self)` – `len(obj)` ishlashi uchun.
- o `__getitem__(self, index)` – `obj[index]` ishlashi uchun.
- o `__setitem__(self, index, value)` – `obj[index] = value` ishlashi uchun.
- o `__delitem__(self, index)` – `del obj[index]` ishlashi uchun.

4. Operatorlarni qo'llash

- o `__add__(self, other)` – `+` operatori.
- o `__sub__(self, other)` – `-` operatori.
- o `__mul__(self, other)` – `*` operatori.
- o `__truediv__(self, other)` – `/` operatori.
- o `__eq__(self, other)` – `==` operatori.
- o `__lt__(self, other)` – `<` operatori.
- o va boshqa matematik va taqqoslash operatorlari.

5. Chaqiriladigan obyektlar

- o `__call__(self, ...)` – obyektни funktsiya kabi chaqirish imkonini beradi.

◆ Misol

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __len__(self):
        return abs(self.x) + abs(self.y)

v1 = Vector(2, 3)
v2 = Vector(4, 5)

print(v1)          # Vector(2, 3)  -> __str__
print(len(v1))     # 5             -> __len__
print(v1 + v2)     # Vector(6, 8)  -> __add__
```

👉 Xullas, **dunder metodlar** obyektlarni odatiy Python funksiyalari va operatorlari bilan mos ishlashini ta'minlaydi.

✓ Amaliy mashqlar

1. `__str__` metodi

Savol:

Book nomli klass yarating. Bu klassda kitob nomi (title) atributi saqlansin. Klassda `__str__` metodini yozing, shunda obyektни `print()` qilinganda kitob nomi ekranga chiqsin.

Javob:

```
class Book:
    def __init__(self, title):
        self.title = title

    def __str__(self):
        return f"Book: {self.title}"

# Sinov
b = Book("Python Asoslari")
print(b)  # Book: Python Asoslari
```

2. `__len__` metodi

Savol:

`Team` nomli klass yarating. Unda `members` nomli ro'yxat (list) bo'lsin. Klassda `__len__` metodini yozing, shunda `len(obj)` chaqirilganda jamoa a'zolari soni qaytib chiqsin.

Javob:

```
class Team:
    def __init__(self, members):
        self.members = members

    def __len__(self):
        return len(self.members)

# Sinov
team = Team(["Ali", "Vali", "Hasan"])
print(len(team))    # 3
```

3. `__add__` metodi

Savol:

`Vector` nomli klass yarating. Har bir vektorda `x` va `y` koordinatalar bo'lsin. Klassda `__add__` metodini yozing, shunda `v1 + v2` bajarilganda yangi vektor koordinatalari (ikkalasi qo'shilib) qaytarilsin.

Javob:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

# Sinov
v1 = Vector(2, 3)
v2 = Vector(5, 7)
print(v1 + v2)    # Vector(7, 10)
```

4. `__getitem__` va `__setitem__` metodlari

Savol:

Grades nomli klass yarating. Unda grades nomli baholar ro'yxati bo'lsin. Klassda `__getitem__` metodini yozing, shunda `obj[0]` orqali birinchi bahoni olish mumkin bo'lsin. `__setitem__` metodini yozing, shunda `obj[1] = 95` orqali bahoni o'zgartirish mumkin bo'lsin.

Javob:

```
class Grades:
    def __init__(self, grades):
        self.grades = grades

    def __getitem__(self, index):
        return self.grades[index]

    def __setitem__(self, index, value):
        self.grades[index] = value

# Sinov
g = Grades([80, 90, 100])
print(g[0])    # 80
g[1] = 95
print(g[1])    # 95
```

🔥 Qo'shimcha maxsus metodlardan misollar

`__eq__` – tenglikni solishtirish

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

p1 = Point(2, 3)
p2 = Point(2, 3)
print(p1 == p2)    # True
```

`__contains__` – in operatori

```
class Bag:
    def __init__(self, items):
        self.items = items

    def __contains__(self, item):
        return item in self.items
```

```
bag = Bag(["apple", "banana", "orange"])
print("apple" in bag)    # True
print("grape" in bag)    # False
```

`__call__` – obyektни funktsiya kabi chaqirish

```
class Greeter:
    def __init__(self, name):
        self.name = name

    def __call__(self):
        return f"Hello, {self.name}!"

g = Greeter("Ali")
print(g())    # Hello, Ali!
```

Klass xususiyatlari (Class Attributes)

Ta'rif:

Klass xususiyatlari — bu **klassning o'ziga tegishli** bo'lgan atributlardir. Ular barcha obyektlar uchun umumiy hisoblanadi. Ya'ni, klassdan nechta obyekt yaratilmasin, hammasi bitta klass xususiyatidan foydalanadi.

 Muhim farq:

- **Klass xususiyati** → Klass darajasida belgilanadi.
 - **Obyekt (instans) xususiyati** → `__init__` metodida `self` orqali yaratiladi.
-

Oddiy misol

```
class Employee:
    company = "OpenAI"    # Klass xususiyati

    def __init__(self, name):
        self.name = name    # Obyekt xususiyati

e1 = Employee("Tom")
e2 = Employee("Alice")

print(e1.company)    # OpenAI
print(e2.company)    # OpenAI
```


🔑 Bu yerda `company` → barcha obyektlar uchun umumiy. `name` esa har bir obyektga alohida saqlanadi.

Klass xususiyatini o‘zgartirish

```
Employee.company = "DeepMind"
```

```
print(e1.company) # DeepMind
print(e2.company) # DeepMind
```

→ Klass xususiyatini **klass nomi orqali** o‘zgartirsak, barcha obyektlarga ta’sir qiladi.

Agar **obyekt orqali** o‘zgartirsak:

```
e1.company = "Test" # faqat e1 uchun yangi instans atribut qo‘shiladi
print(e1.company)   # Test
print(e2.company)   # DeepMind
```

→ Bunda `e1` uchun alohida `company` atributi yaratiladi, umumiy klass xususiyati o‘zgarmaydi.

✓ Amaliy mashqlar

1. `Car` klassi yarating. Unda **klass xususiyati** sifatida `wheels = 4` bo‘lsin (barcha mashinalarda 4 g‘ildirak). Har bir obyekt `name` atributini olsun. Bir nechta obyekt yaratib, ularning nomi va g‘ildiraklar sonini chiqaring.

Javob:

```
class Car:
    wheels = 4 # klass xususiyati

    def __init__(self, name):
        self.name = name # obyekt xususiyati

# Sinov
c1 = Car("BMW")
c2 = Car("Mercedes")

print(c1.name, c1.wheels) # BMW 4
print(c2.name, c2.wheels) # Mercedes 4
```

2. `BankAccount` klassi

Savol:

BankAccount klassida **klass xususiyati** sifatida `bank_name = "National Bank"` belgilang. Har bir obyekt `owner` va `balance` atributlariga ega bo'lsin. Bir nechta obyekt yarating va ularning ma'lumotlarini chiqaring.

Javob:

```
class BankAccount:
    bank_name = "National Bank"

    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

# Sinov
a1 = BankAccount("Ali", 1000)
a2 = BankAccount("Vali", 500)

print(a1.owner, a1.balance, a1.bank_name)    # Ali 1000 National Bank
print(a2.owner, a2.balance, a2.bank_name)    # Vali 500 National Bank
```

3. Klass xususiyatini obyekt orqali o'zgartirish

Savol:

Employee klassini yarating. Klass xususiyati sifatida `company = "OpenAI"` belgilang. Ikki obyekt yarating. Keyin birinchi obyekt orqali `company` ni "Test" ga o'zgartiring va natijani kuzating.

Javob:

```
class Employee:
    company = "OpenAI"

    def __init__(self, name):
        self.name = name

# Sinov
e1 = Employee("Tom")
e2 = Employee("Alice")

e1.company = "Test"    # Faqat e1 uchun yangi instans xususiyati
print(e1.name, e1.company)    # Tom Test
print(e2.name, e2.company)    # Alice OpenAI
print(Employee.company)    # OpenAI
```

 Xulosa: obyekt orqali klass xususiyatini o'zgartirsak, aslida yangi instans atributi yaratiladi, klass xususiyati o'zgarmaydi.

4. `Student` klassida `count` nomli klass xususiyatini yarating. Har yangi obyekt yaratilganda `count` 1 ga oshib borsin. Nechta obyekt yaratilganini ko'rsating.

Javob:

```
class Student:
    count = 0 # Klass xususiyati

    def __init__(self, name):
        self.name = name
        Student.count += 1 # Har yangi obyekt yaratilganda +1

# Sinov
s1 = Student("Ali")
s2 = Student("Vali")
s3 = Student("Hasan")

print("Jami studentlar soni:", Student.count) # 3
```



Xulosa:

- Klass xususiyatlari barcha obyektlarga umumiy.
- Klass nomi orqali o'zgartirilsa → hamma obyektga ta'sir qiladi.
- Obyekt orqali o'zgartirilsa → faqat o'sha obyekt uchun yangi atribut hosil bo'ladi.
- Hisoblagich (`count`) kabi umumiy ma'lumotlarni kuzatishda juda qulay.

4. Dekoratorlar (Decorators)

Ta'rif:

Dekorator — bu boshqa funksiya yoki metodni qabul qilib, uning xatti-harakatini o'zgartiruvchi maxsus funksiya yoki yozuvdir. Klasslarda asosan quyidagi dekoratorlar ishlatiladi:

1. **@property** – metodni oddiy atribut kabi ishlatish imkonini beradi. Odatda faqat o'qiladigan qiymatlar uchun ishlatiladi.
2. **@classmethod** – obyektlarga emas, balki klassning o'ziga tegishli bo'ladi. Ko'pincha obyekt yaratishning qo'shimcha usullari uchun ishlatiladi.
3. **@staticmethod** – klassga aloqador, lekin obyekt yoki klass atributlariga bog'liq bo'lmagan metodlarni yozishda ishlatiladi.



Amaliy mashqlar

1. @property

Savol:

Circle klassini yarating. Unda radius atributi bo'lsin. area (yuza) faqat o'qish uchun property sifatida yaratilishi kerak (πr^2 formulasi asosida).

Javob (kod):

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return math.pi * (self.radius ** 2)

c1 = Circle(5)
print("Radius:", c1.radius)
print("Yuza:", c1.area)  # faqat o'qish mumkin
```

2. @classmethod

Savol:

Person klassida umumiy hisoblagich (count) bo'lsin. Har safar yangi obyekt yaratilganda count +1 oshsin. get_count() klass metodi orqali nechta obyekt yaratilganini ko'rsatish kerak.

Javob (kod):

```
class Person:
    count = 0  # class attribute

    def __init__(self, name):
        self.name = name
        Person.count += 1

    @classmethod
    def get_count(cls):
        return cls.count

p1 = Person("Tom")
p2 = Person("Alice")
print("Obyektlar soni:", Person.get_count())
```

3. @staticmethod

Savol:

Math klassini yarating. Unda `add(a, b)` static metodini yozing. Obyekt yaratmasdan turib uni chaqiring.

Javob (kod):

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b

print("Yig'indi:", Math.add(10, 20))
```

4. Birlashtirish

Savol:

Employee klassini yarating. Unda:

- `company` (klass xususiyati) bo'lsin
- `fullname` (property) bo'lsin
- `from_string()` (classmethod) bo'lsin, u "Ism Familiya" ko'rinishidagi matndan obyekt yaratsin
- `is_adult(age)` (staticmethod) bo'lsin, u yoshni tekshirib, True/False qaytarsin

Turli obyektlar orqali ishlashini ko'rsating.

Javob (kod):

```
class Employee:
    company = "OpenAI"  # class attribute

    def __init__(self, first, last, age):
        self.first = first
        self.last = last
        self.age = age

    @property
    def fullname(self):
        return f"{self.first} {self.last}"

    @classmethod
    def from_string(cls, emp_str):
        first, last, age = emp_str.split()
        return cls(first, last, int(age))

    @staticmethod
    def is_adult(age):
        return age >= 18
```

```
# Obyekt yaratish odatdagi usulda
e1 = Employee("Tom", "Smith", 25)
print(e1.fullname, "| Kompaniya:", e1.company)

# Obyekt yaratish classmethod orqali
e2 = Employee.from_string("Alice Johnson 17")
print(e2.fullname, "| Yoshi:", e2.age)

# staticmethod ishlatish
print("Tom kattami?", Employee.is_adult(e1.age))
print("Alice kattami?", Employee.is_adult(e2.age))
```

1. Vorislik (Inheritance)

Ta'rif:

Vorislik – mavjud klass asosida yangi klass yaratish mexanizmi. Bola klass (child) ota klass (parent) metod va atributlarini meros qilib oladi.

Sintaksis:

```
class Parent:
    pass

class Child(Parent):
    pass
```

Afzalliklari:

- Kodni qayta ishlatish imkoniyati
- Real hayotdagi ierarxiyalarni yaratish
- Metodlarni qayta aniqlash (override)
- Polimorfizm

Misol:

```
class Person:
    def __init__(self, name):
        self.name = name
    def display(self):
        print("Name:", self.name)

class Employee(Person):
    def work(self):
        print(self.name, "works")
```

2. Maxsus metodlar (Dunder methods)

Ta'rif:

Maxsus metodlar ikki pastki chiziq bilan boshlanadi va tugaydi (`__init__`, `__str__`, `__len__`, va h.k.).

Asosiylari:

- `__init__` → konstruktor
- `__str__` → `print(obj)` uchun

- `__len__` → `len(obj)` uchun
 - `__getitem__`, `__setitem__` → indeks bilan ishlash
 - `__eq__`, `__lt__` → taqqoslash
 - `__add__` → qo'shish
 - `__call__` → obyektни funktsiya kabi chaqirish
 - `__contains__` → `in` operatori
-

3. Klass xususiyatlari (Class Attributes)

Ta'rif:

Klass xususiyatlari → klassning o'ziga tegishli, barcha obyektlar uchun umumiy.

Farqi:

- Klass xususiyati → klass nomi orqali belgilanadi
- Instans xususiyati → obyektga tegishli (`self` orqali)

Misol:

```
class Employee:
    company = "OpenAI" # class attribute
    def __init__(self, name):
        self.name = name # instance attribute
```

4. Dekoratorlar (Decorators)

Ta'rif:

Dekorator – metod yoki funktsiyaning xatti-harakatini o'zgartiruvchi maxsus yozuv.

Asosiylari:

- `@property` → metodni atribut sifatida ishlatish
- `@classmethod` → klassga tegishli metod
- `@staticmethod` → obyektga bog'liq bo'lmagan metod



Xulosa

- **Vorislilik** → klasslar ierarxiasini yaratadi
- **Maxsus metodlar** → obyektlarni qulay ishlatish imkonini beradi (`len`, `+`, `print` va h.k.)
- **Klass xususiyatlari** → umumiy ma'lumotni saqlash uchun
- **Dekoratorlar** → metodlarning turli xatti-harakatlarini belgilash